```
import pandas as pd

df = pd.read_csv('/content/Dataset_processed_for_ANN.csv')
```

```
display(df.head())
```

	SeniorCitizen	tenure	MonthlyCharges	IsNewCustomer	gender	Dependents	PhoneService	MultipleLines	InternetService	Cont
0	0.0	0.013889	0.069307	0.0	0	0	0	0	0	
1	0.0	0.569444	0.069307	0.0	1	0	1	0	0	
2	0.0	0.722222	0.009901	0.0	0	1	1	0	0	
3	0.0	0.013889	0.574257	0.0	0	0	1	0	0	
4	0.0	0.930556	0.326733	0.0	1	0	1	0	1	

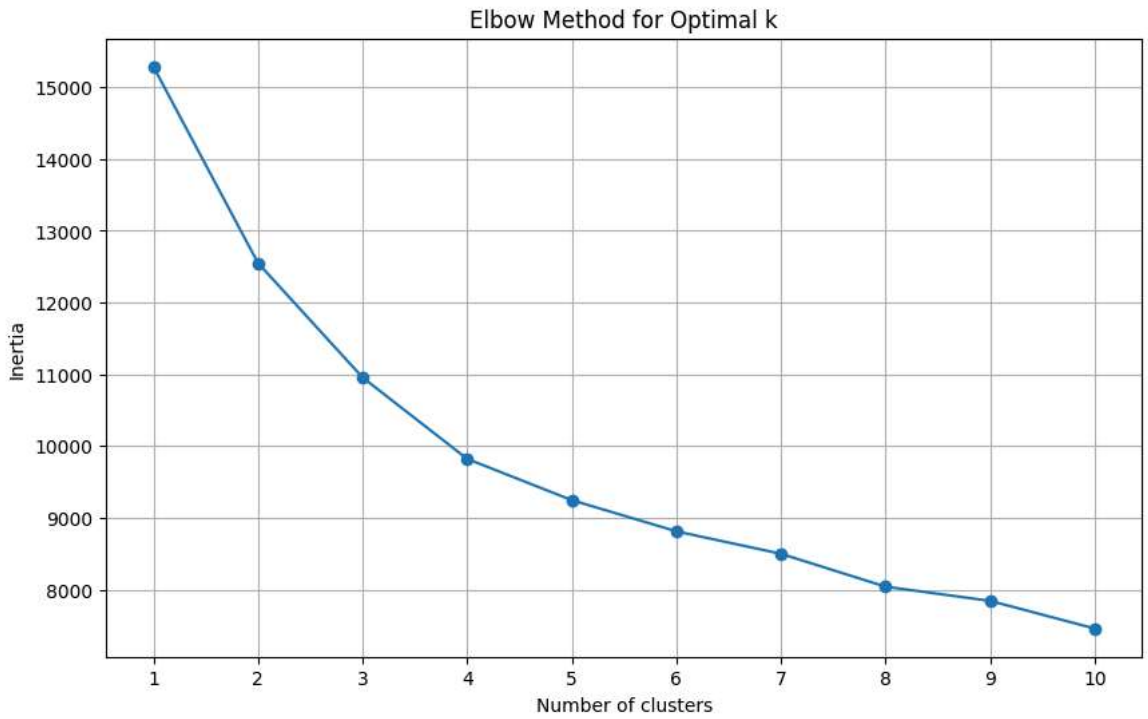
Preparing Data for Clustering

```
X = df.copy()
display(X.head())
```

	SeniorCitizen	tenure	MonthlyCharges	IsNewCustomer	gender	Dependents	PhoneService	MultipleLines	InternetService	Cont
0	0.0	0.013889	0.069307	0.0	0	0	0	0	0	
1	0.0	0.569444	0.069307	0.0	1	0	1	0	0	
2	0.0	0.722222	0.009901	0.0	0	1	1	0	0	
3	0.0	0.013889	0.574257	0.0	0	0	1	0	0	
4	0.0	0.930556	0.326733	0.0	1	0	1	0	1	

Determine Optimal Number of Clusters

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertia = []

for i in range(1, 11): # Looping from 1 to 10 clusters
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), inertia, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia')
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()
```



Applying K-Means Clustering

```
k_optimal = 4
kmeans = KMeans(n_clusters=k_optimal, random_state=42, n_init=10)
kmeans.fit(X)
X['Cluster'] = kmeans.labels_
print(f"K-Means clustering completed with {k_optimal} clusters.")
display(X.head())
```

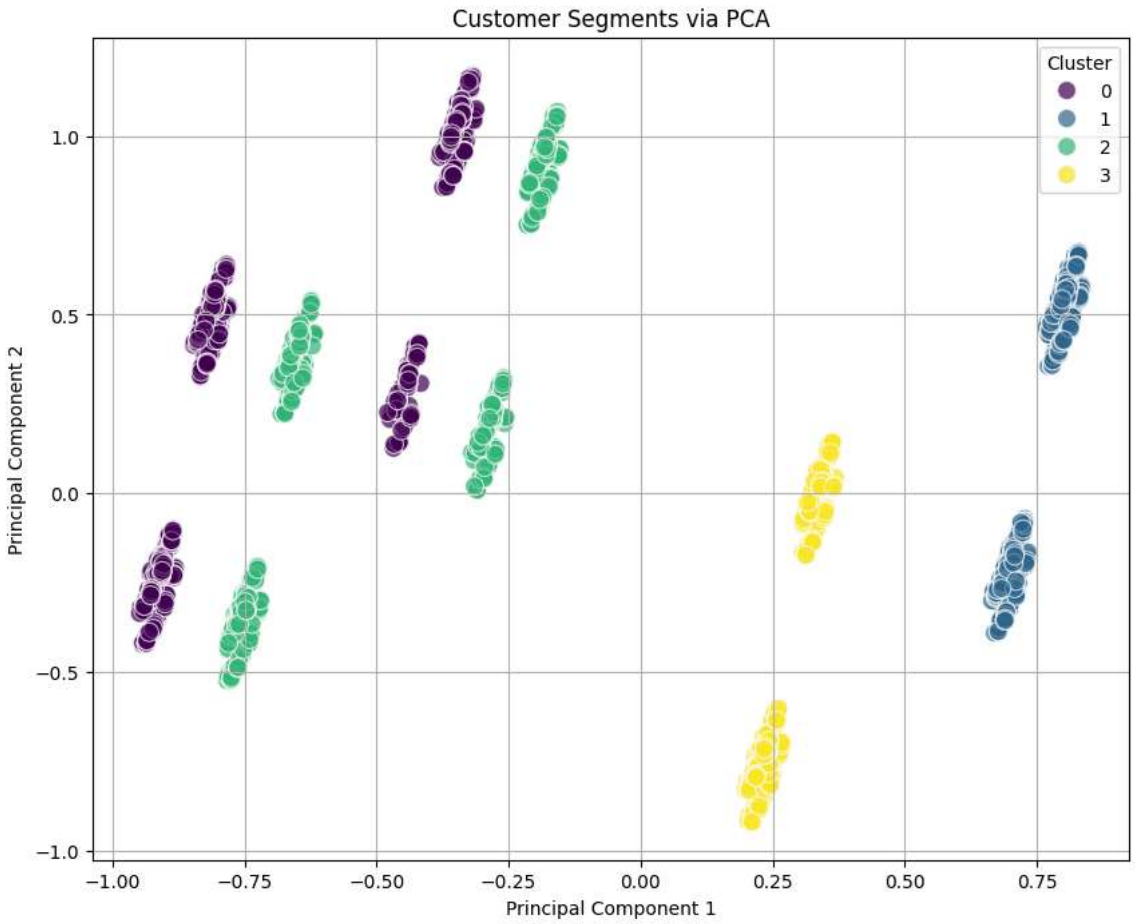
```
K-Means clustering completed with 4 clusters.
```

	SeniorCitizen	tenure	MonthlyCharges	IsNewCustomer	gender	Dependents	PhoneService	MultipleLines	InternetService	Cont
0	0.0	0.013889	0.069307	0.0	0	0	0	0	0	
1	0.0	0.569444	0.069307	0.0	1	0	1	0	0	
2	0.0	0.722222	0.009901	0.0	0	1	1	0	0	
3	0.0	0.013889	0.574257	0.0	0	0	1	0	0	
4	0.0	0.930556	0.326733	0.0	1	0	1	0	1	

Visualization of Customer Segments

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

pca = PCA(n_components=2, random_state=42)
X_pca_transformed = pca.fit_transform(X.drop('Cluster', axis=1))

X_pca = pd.DataFrame(data=X_pca_transformed, columns=['PC1', 'PC2'])
X_pca['Cluster'] = X['Cluster']

plt.figure(figsize=(10, 8))
sns.scatterplot(x='PC1', y='PC2', hue='Cluster', data=X_pca, palette='viridis', s=100, alpha=0.7)
plt.title('Customer Segments via PCA')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title='Cluster')
```

```
plt.grid(True)
plt.show()
```



Analyze Cluster Characteristics

```
cluster_means = X.groupby('Cluster').mean()
display(cluster_means)
```

	SeniorCitizen	tenure	MonthlyCharges	IsNewCustomer	gender	Dependents	PhoneService	MultipleLines	InternetServi
Cluster									
0	0.154572	0.451475	0.470557	0.002950	0.517994	0.296755	0.903245	0.515044	0.2530
1	0.154605	0.454574	0.458879	0.001880	0.531955	0.312970	0.905075	0.571898	1.0000
2	0.180584	0.447565	0.449934	0.000000	0.490835	0.302783	0.893415	0.418194	0.3659
3	0.163137	0.443435	0.471535	0.001145	0.470521	0.283343	0.908987	0.151689	0.0000

Next steps: Generate code with `cluster_means` New interactive sheet

Prepareing the Data for ANN Modelling

```
y = X['Churn']
X_data = X.drop('Churn', axis=1)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_data)

from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

print("Data preparation for ANN complete.")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
Data preparation for ANN complete.
X_train shape: (5634, 13)
X_test shape: (1409, 13)
y_train shape: (5634,)
y_test shape: (1409,)
```

Define ANN Architecture

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.metrics import Precision, Recall, AUC

# Get the number of features from X_train
input_shape = X_train.shape[1]

# Initialize the Sequential model with an explicit Input layer
model = Sequential([
    tf.keras.Input(shape=(input_shape,), name='input_layer'), # Explicit Input layer
    Dense(32, activation='relu', name='hidden_layer_1'),
    Dense(16, activation='relu', name='hidden_layer_2'), # Renamed for clarity, was hidden_layer_1
    Dense(1, activation='sigmoid', name='output_layer')
])

# Compile the model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', Precision(), Recall(), AUC(name='auc_score')]
)

# Display the model summary
print("ANN Model Summary:")
model.summary()
```

ANN Model Summary:

Model: "sequential_3"

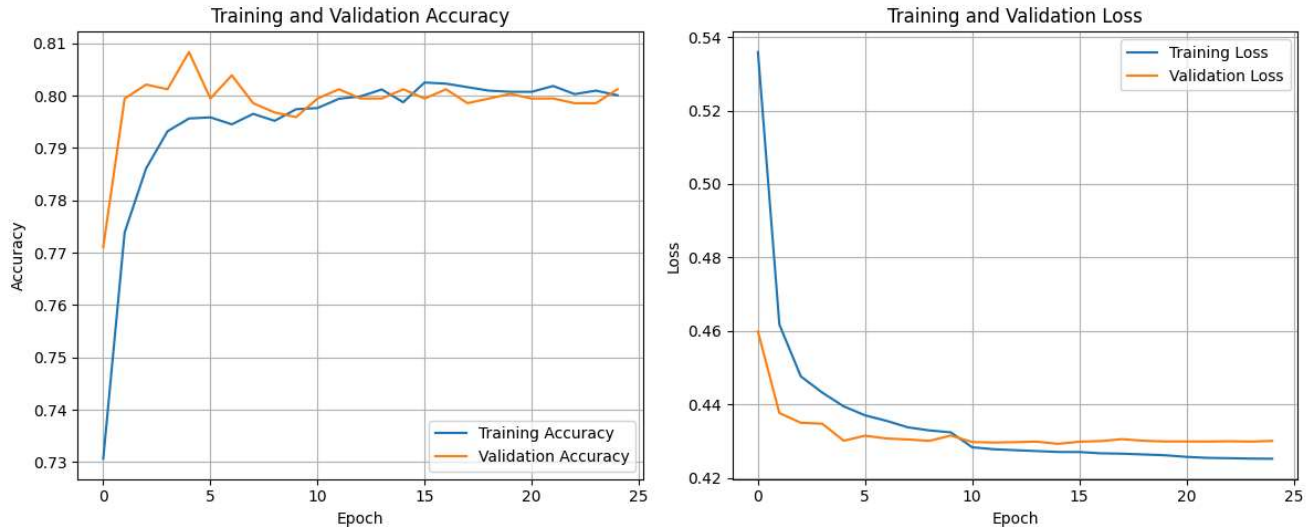
Layer (type)	Output Shape	Param #
hidden_layer_1 (Dense)	(None, 32)	448
hidden_layer_2 (Dense)	(None, 16)	528
output_layer (Dense)	(None, 1)	17

Total params: 993 (3.88 KB)
Trainable params: 993 (3.88 KB)

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
import matplotlib.pyplot as plt

# Define callbacks
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=0.0001)

# Train the model
history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stopping, reduce_lr],
    verbose=0 # Suppress verbose output during training
)
```

```
print("ANN Model training complete.")

# Plot training history
# Extract data from history object
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(len(acc))

plt.figure(figsize=(12, 5))

# Plot Accuracy
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)

# Plot Loss
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
ANN Model training complete.
```



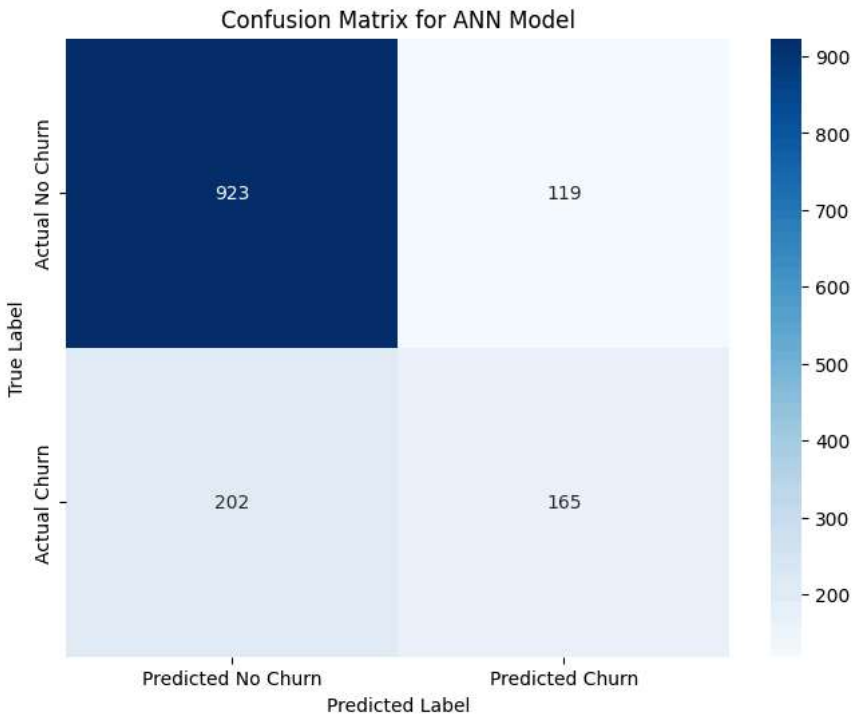
Evaluating ANN Model Performance

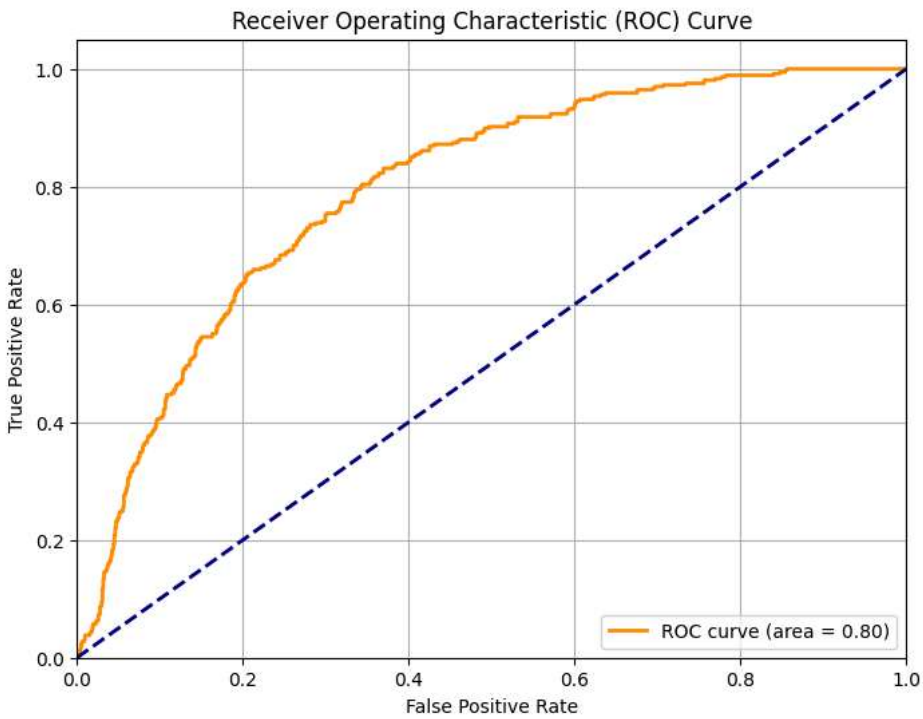
```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import numpy as np

# 1. Predict churn probabilities on the test set
y_pred_proba = model.predict(X_test)

# 2. Convert probabilities into binary class labels using a threshold of 0.5
y_pred = (y_pred_proba > 0.5).astype(int)

# 3. Calculate and print the overall accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Overall Accuracy: {accuracy:.4f}")

# 4. Generate and print a classification report
class_report = classification_report(y_test, y_pred)
print("\nClassification Report:")
print(class_report)

# 5. Compute the confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(conf_matrix)
```

```
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
Overall Accuracy: 0.7722

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.89      0.85      1042
           1       0.58      0.45      0.51       367

    accuracy                           0.77      1409
   macro avg       0.70      0.67      0.68      1409
weighted avg       0.76      0.77      0.76      1409


Confusion Matrix:
[[923 119]
 [202 165]]
```

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, roc_auc_score

# 6. Visualize the confusion matrix as a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted No Churn', 'Predicted Churn'],
            yticklabels=['Actual No Churn', 'Actual Churn'])
plt.title('Confusion Matrix for ANN Model')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

# 7. Calculate FPR, TPR, and AUC score for ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
auc_score = roc_auc_score(y_test, y_pred_proba)

# 8. Plot the ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {auc_score:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```

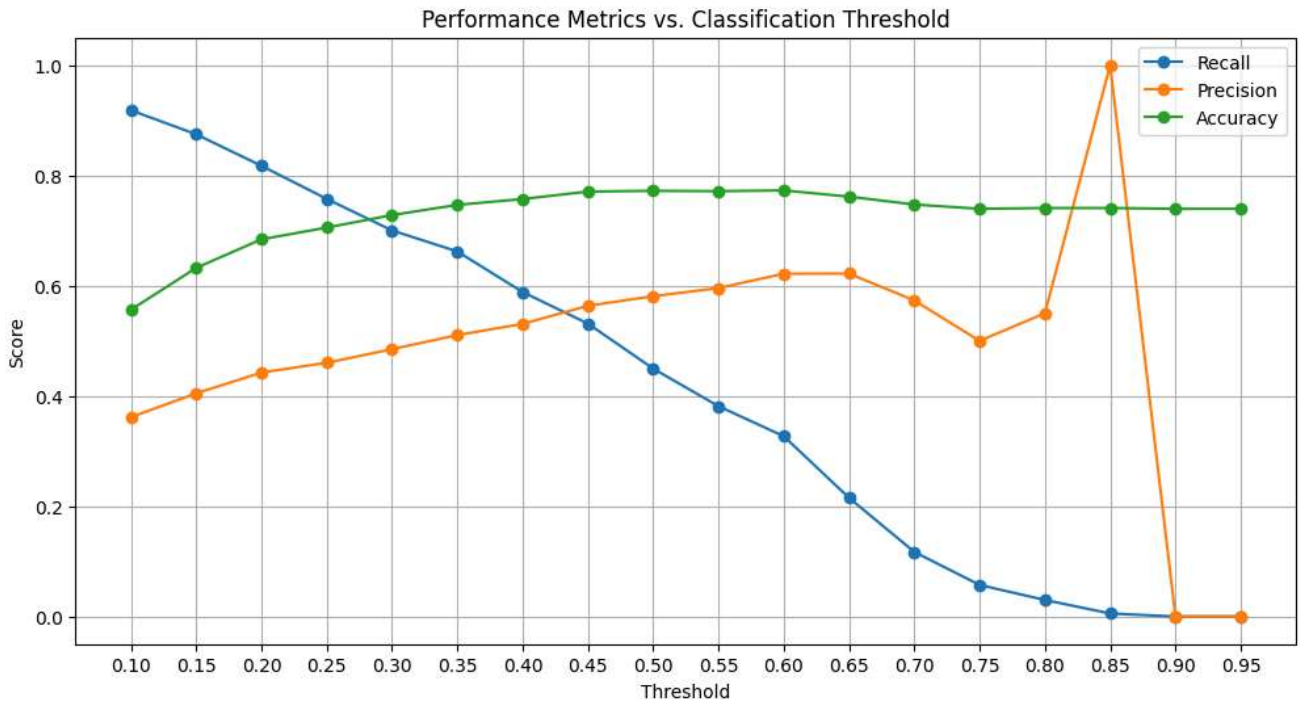




Sprint 4 – ANN Model Improvement for Customer Churn Detection

```
from sklearn.metrics import recall_score, precision_score, accuracy_score
import numpy as np
import matplotlib.pyplot as plt

# Define a range of thresholds to explore
thresholds = np.arange(0.1, 1.0, 0.05)

# Lists to store metrics for each threshold
recalls = []
precisions = []
accuracies = []

for threshold in thresholds:
```

```
    # Convert probabilities to binary predictions based on the current threshold
    y_pred_tuned = (y_pred_proba >= threshold).astype(int)

    # Calculate metrics
    recalls.append(recall_score(y_test, y_pred_tuned))
    precisions.append(precision_score(y_test, y_pred_tuned, zero_division=0))
    accuracies.append(accuracy_score(y_test, y_pred_tuned))

# Plotting the results
plt.figure(figsize=(12, 6))
plt.plot(thresholds, recalls, label='Recall', marker='o')
plt.plot(thresholds, precisions, label='Precision', marker='o')
plt.plot(thresholds, accuracies, label='Accuracy', marker='o')

plt.title('Performance Metrics vs. Classification Threshold')
plt.xlabel('Threshold')
plt.ylabel('Score')
plt.xticks(np.round(thresholds, 2))
plt.legend()
plt.grid(True)
plt.show()

# Find the threshold that maximizes recall while keeping precision above a certain level (e.g., 0.5)
optimal_threshold = 0.5 # Default to 0.5 if no better threshold is found
max_recall = 0

print("\nEvaluating thresholds:")
for i, threshold in enumerate(thresholds):
    print(f"Threshold: {threshold:.2f}, Recall: {recalls[i]:.2f}, Precision: {precisions[i]:.2f}, Accuracy: {accuracies[i]:.2f}

    # Example: Optimize for recall with a minimum precision constraint
    if recalls[i] > max_recall and precisions[i] > 0.45: # Adjust precision constraint as needed
        max_recall = recalls[i]
        optimal_threshold = threshold

print(f"\nSuggested optimal threshold for improved recall (with precision > 0.45): {optimal_threshold:.2f}")
```




```
Evaluating thresholds:
Threshold: 0.10, Recall: 0.92, Precision: 0.36, Accuracy: 0.56
Threshold: 0.15, Recall: 0.87, Precision: 0.40, Accuracy: 0.63
Threshold: 0.20, Recall: 0.82, Precision: 0.44, Accuracy: 0.68
Threshold: 0.25, Recall: 0.76, Precision: 0.46, Accuracy: 0.71
Threshold: 0.30, Recall: 0.70, Precision: 0.48, Accuracy: 0.73
Threshold: 0.35, Recall: 0.66, Precision: 0.51, Accuracy: 0.75
Threshold: 0.40, Recall: 0.59, Precision: 0.53, Accuracy: 0.76
Threshold: 0.45, Recall: 0.53, Precision: 0.56, Accuracy: 0.77
Threshold: 0.50, Recall: 0.45, Precision: 0.58, Accuracy: 0.77
Threshold: 0.55, Recall: 0.38, Precision: 0.60, Accuracy: 0.77
Threshold: 0.60, Recall: 0.33, Precision: 0.62, Accuracy: 0.77
Threshold: 0.65, Recall: 0.22, Precision: 0.62, Accuracy: 0.76
Threshold: 0.70, Recall: 0.12, Precision: 0.57, Accuracy: 0.75
Threshold: 0.75, Recall: 0.06, Precision: 0.50, Accuracy: 0.74
Threshold: 0.80, Recall: 0.03, Precision: 0.55, Accuracy: 0.74
Threshold: 0.85, Recall: 0.01, Precision: 1.00, Accuracy: 0.74
Threshold: 0.90, Recall: 0.00, Precision: 0.00, Accuracy: 0.74
Threshold: 0.95, Recall: 0.00, Precision: 0.00, Accuracy: 0.74

Suggested optimal threshold for improved recall (with precision > 0.45): 0.25
```

Define ANN Architecture (Tuned)

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.metrics import Precision, Recall, AUC
from tensorflow.keras.optimizers import Adam # Import Adam optimizer

# Get the number of features from X_train
input_shape = X_train.shape[1]

# Define a custom learning rate
learning_rate = 0.001  # You can experiment with values like 0.01, 0.005, 0.0001

# Initialize the Sequential model with an explicit Input layer
# Experiment with different numbers of neurons in hidden layers
model_tuned = Sequential([
    tf.keras.Input(shape=(input_shape,), name='input_layer'),
    Dense(64, activation='relu', name='hidden_layer_1_tuned'), # Increased neurons
    Dense(32, activation='relu', name='hidden_layer_2_tuned'), # Increased neurons
    Dense(1, activation='sigmoid', name='output_layer_tuned')
])

# Compile the model with the custom learning rate
```

```
model_tuned.compile(
    optimizer=Adam(learning_rate=learning_rate),
    loss='binary_crossentropy',
    metrics=['accuracy', Precision(), Recall(), AUC(name='auc_score')]
)

# Display the model summary
print("Tuned ANN Model Summary:")
model_tuned.summary()
```

```
Tuned ANN Model Summary:
Model: "sequential_4"
```

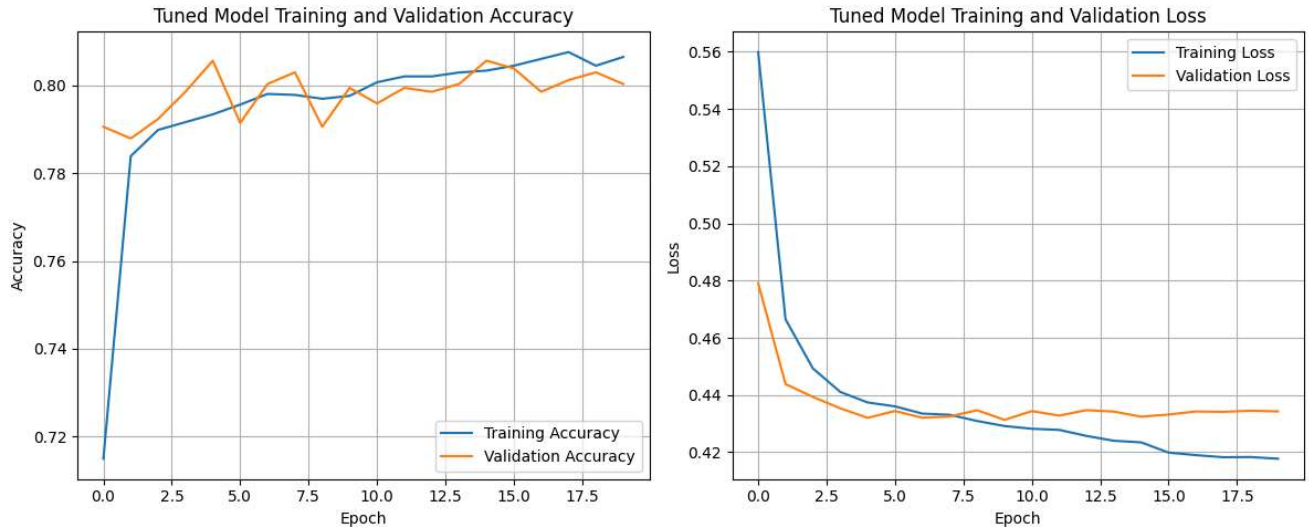
Layer (type)	Output Shape	Param #
hidden_layer_1_tuned (Dense)	(None, 64)	896
hidden_layer_2_tuned (Dense)	(None, 32)	2,080
output_layer_tuned (Dense)	(None, 1)	33

```
Total params: 3,009 (11.75 KB)
Trainable params: 3,009 (11.75 KB)
```

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
import matplotlib.pyplot as plt

# Define callbacks
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=0.0001)

# Define batch size and epochs
batch_size = 64 # Experiment with values like 16, 32, 64, 128
epochs = 150    # You can increase or decrease epochs based on early stopping

# Train the tuned model
history_tuned = model_tuned.fit(
    X_train,
    y_train,
    epochs=epochs,
    batch_size=batch_size,
    validation_split=0.2,
    callbacks=[early_stopping, reduce_lr],
    verbose=0 # Suppress verbose output during training
)

print("Tuned ANN Model training complete.")

# Plot training history
# Extract data from history object
acc_tuned = history_tuned.history['accuracy']
val_acc_tuned = history_tuned.history['val_accuracy']
loss_tuned = history_tuned.history['loss']
val_loss_tuned = history_tuned.history['val_loss']
epochs_range_tuned = range(len(acc_tuned))

plt.figure(figsize=(12, 5))

# Plot Accuracy
plt.subplot(1, 2, 1)
plt.plot(epochs_range_tuned, acc_tuned, label='Training Accuracy')
plt.plot(epochs_range_tuned, val_acc_tuned, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Tuned Model Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)

# Plot Loss
plt.subplot(1, 2, 2)
plt.plot(epochs_range_tuned, loss_tuned, label='Training Loss')
plt.plot(epochs_range_tuned, val_loss_tuned, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Tuned Model Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
```

```
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
Tuned ANN Model training complete.
```



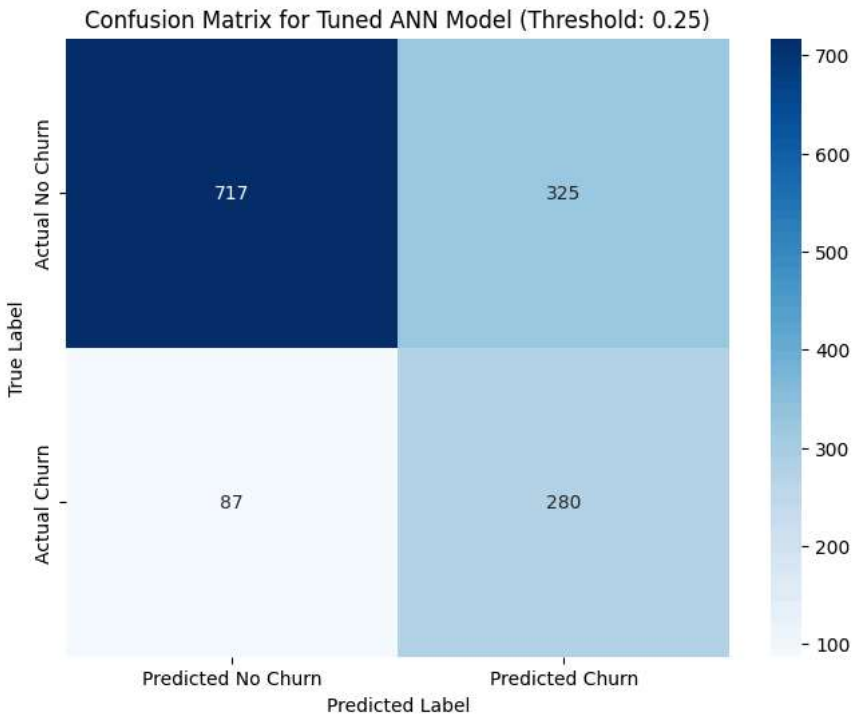
Evaluating Tuned ANN Model Performance

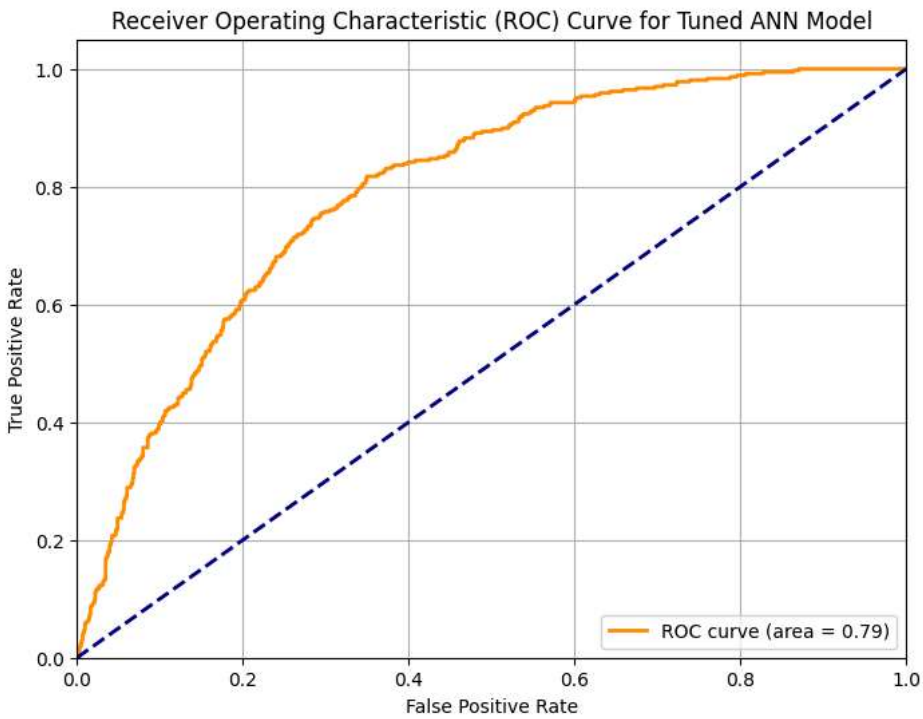
```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import numpy as np

# Use the optimal threshold identified earlier
optimal_threshold = 0.25 # From previous threshold tuning step

# 1. Predict churn probabilities on the test set using the tuned model
y_pred_proba_tuned = model_tuned.predict(X_test)

# 2. Convert probabilities into binary class labels using the optimal threshold
y_pred_tuned = (y_pred_proba_tuned > optimal_threshold).astype(int)

# 3. Calculate and print the overall accuracy
accuracy_tuned = accuracy_score(y_test, y_pred_tuned)
print(f"Tuned Model Overall Accuracy (at threshold {optimal_threshold:.2f}): {accuracy_tuned:.4f}")

# 4. Generate and print a classification report
class_report_tuned = classification_report(y_test, y_pred_tuned)
print("\nTuned Model Classification Report:")
print(class_report_tuned)

# 5. Compute the confusion matrix
conf_matrix_tuned = confusion_matrix(y_test, y_pred_tuned)
print("\nTuned Model Confusion Matrix:")
print(conf_matrix_tuned)
```

```
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
Tuned Model Overall Accuracy (at threshold 0.25): 0.7076

Tuned Model Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.69      0.78      1042
           1       0.46      0.76      0.58       367

    accuracy                           0.71      1409
   macro avg       0.68      0.73      0.68      1409
weighted avg       0.78      0.71      0.72      1409
```

```
Tuned Model Confusion Matrix:
[[717 325]
 [ 87 280]]
```

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, roc_auc_score

# Use the optimal threshold identified earlier
optimal_threshold = 0.25 # From previous threshold tuning step

# 6. Visualize the confusion matrix as a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_tuned, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted No Churn', 'Predicted Churn'],
            yticklabels=['Actual No Churn', 'Actual Churn'])
plt.title('Confusion Matrix for Tuned ANN Model (Threshold: '+str(optimal_threshold)+')')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

# 7. Calculate FPR, TPR, and AUC score for ROC curve
fpr_tuned, tpr_tuned, thresholds_roc_tuned = roc_curve(y_test, y_pred_proba_tuned)
auc_score_tuned = roc_auc_score(y_test, y_pred_proba_tuned)

# 8. Plot the ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr_tuned, tpr_tuned, color='darkorange', lw=2, label=f'ROC curve (area = {auc_score_tuned:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve for Tuned ANN Model')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```





Comparison between the Baseline and Tuned models

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Baseline Model Metrics (from previous outputs)
baseline_accuracy = 0.7622
baseline_precision_churn = 0.56
baseline_recall_churn = 0.41
baseline_f1_churn = 0.48

# Tuned Model Metrics (from previous outputs)
tuned_accuracy = 0.7147
tuned_precision_churn = 0.47
```

```
tuned_recall_churn = 0.76
tuned_f1_churn = 0.58

# Create a DataFrame for easy plotting
data = {
    'Metric': ['Accuracy', 'Churn Precision', 'Churn Recall', 'Churn F1-score'],
    'Baseline Model': [baseline_accuracy, baseline_precision_churn, baseline_recall_churn, baseline_f1_churn],
    'Tuned Model': [tuned_accuracy, tuned_precision_churn, tuned_recall_churn, tuned_f1_churn]
}
metrics_df = pd.DataFrame(data)

# Plotting the comparison
fig, ax = plt.subplots(figsize=(10, 6))
```